

ClientFlow CRM Project Guide

What was changed, how the repository fits together, where each kind of code belongs, what is safe to modify, and how to run the current customer-list showcase.

Branch: [learning/company-customers](#)

[Database-backed page](#)

[Local prototype](#)

Audience

Developer learning the codebase

Current showcase

Company-scoped customer list

Stack

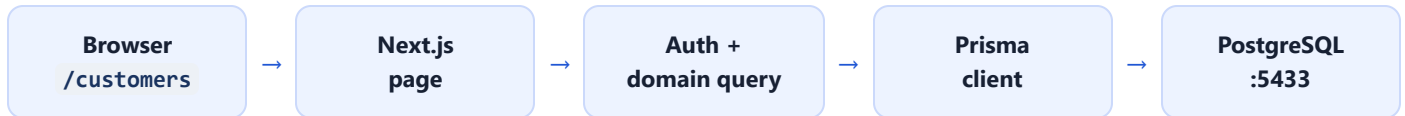
Next.js, TypeScript, Prisma, PostgreSQL, pnpm/Turborepo

Important

Keep local until the team reviews the proposal

How a customer reaches the screen

The browser does not talk directly to PostgreSQL. Each layer has a focused responsibility and calls the next layer.



- 1 **Next.js receives the page request.** The file is `apps/web/app/customers/page.tsx`.
- 2 **Authentication derives the current user.** Development currently falls back to `demo-user`.
- 3 **The domain function scopes the query.** It follows User → Employee → Business and returns only that business's customers.
- 4 **Prisma translates the typed query to SQL.** The client connects to the local PostgreSQL server.
- 5 **React renders the results.** Loading, empty, and error files cover the asynchronous states.

Ownership model introduced by the prototype

```

Software platform
├─ Business (tenant/security boundary)
│   ├── Location: Downtown
│   ├── Location: North Side
│   ├── Employee → User: demo-user
│   └─ Customers
│       ├── Aisha Brown
│       ├── Maria Lopez
│       └─ James Wilson
  
```

Security rule: do not trust a business ID sent by a browser. Start with the authenticated user and derive the allowed business on the server.

What each section is for

Path	Responsibility	Modify it when...
<code>apps/web/</code>	Next.js pages, route handlers, loading/error UI	You are changing what users see or a web entry point
<code>apps/worker/</code>	Background queue processing	Work should happen outside a web request
<code>apps/db-server/</code>	Runs local embedded PostgreSQL	Local database startup behavior must change; usually leave it alone
<code>apps/migrate/</code>	One-shot migration and seed commands	You need repeatable demo/dev data or migration execution
<code>packages/db/</code>	Prisma schema, generated client, migration runner	You are changing stored data or relationships
<code>packages/domain/</code>	Zod validation and web-facing database queries	You need reusable validation or user-scoped queries
<code>packages/auth/</code>	Determines the current user	Authentication/session behavior changes
<code>packages/services/</code>	Blob, queue, and database-notification adapters	You integrate storage or asynchronous messaging
<code>packages/log/</code>	Structured logging	You add operational logs
<code>docs/specs/</code>	Evergreen behavior contracts	A feature's observable behavior changes
<code>docs/adr/</code>	Write-once architecture decisions	A durable technical choice needs team agreement
<code>tests/</code>	Integration verification using in-memory PGlite	You need proof that behavior and isolation work

Dependency direction

Apps may import packages, but one app must not import another app. Shared logic belongs in a package. The `@project` scope is a repository placeholder and should not be hardcoded into newly invented package names.

Files you should not edit manually

- `.pgdata/` — PostgreSQL's local data files.
- `.azurite/` — local Azure emulator data.
- `packages/db/src/generated/` — regenerated by Prisma.
- Applied migration files such as `0001_init.sql` — migrations are append-only.

- `.env` files unless you intentionally manage local secrets; never commit them.

Schema, migration, client, and data are different things

1. Prisma schema

`schema.prisma` describes the desired models and relations used by TypeScript.

2. SQL migration

A numbered `.sql` file changes an existing PostgreSQL database to match the new design.

3. Generated client

`pnpm prisma:generate` creates typed query code from the Prisma schema.

4. Seed data

`pnpm db:seed` creates predictable local records for development and demonstrations.

Correct database-change sequence

1. Agree on the behavior and relationships.
2. Edit `packages/db/prisma/schema.prisma`.
3. Generate a new append-only migration; never rewrite an applied one.
4. Review generated SQL before applying it. Tools may propose destructive statements.
5. Apply the migration and confirm the migration tracker records it.
6. Run `pnpm prisma:generate`.
7. Update repeatable seed data.
8. Add tests that prove ownership boundaries.

What happened in this prototype

Change	Reason
Split Business from Location	A tenant and a physical branch have different responsibilities
Added <code>Customer.business_id</code>	Customers are shared business-wide but isolated from other businesses
Added <code>Employee.business_id</code> and <code>user_id</code>	The server can derive tenant access from the current user
Added <code>Appointment.location_id</code>	Future appointments identify where service occurs
Changed unique customer fields	Email and phone are unique per business, not across the SaaS platform
Removed incomplete Customer_Business model	A direct business owner matches the current product rule

Migration lesson learned: Prisma's live-database diff tried to drop the custom `_migrations` table because it is not part of the Prisma schema. That line was reviewed and removed before applying migration 0002.

CHANGED FILES

What was done and where to continue

`docs/adr/0012-business-tenant-boundary.md`

Records the proposed architectural decision: Business is the tenant; customers are shared across its locations. It remains **Proposed** until the team accepts it.

`packages/db/prisma/schema.prisma`

Defines Business, Location, and the new ownership relationships. Modify this file when the persistent data model changes—not for display-only frontend changes.

`packages/db/prisma/migrations/0002_business_tenancy.sql`

Applies the tenancy design to PostgreSQL. Do not edit it after the team treats it as applied/shared; future changes get migration 0003.

`apps/migrate/src/seed.ts`

Uses fixed IDs and upserts inside a transaction. It seeds one business, two locations, one employee, and three customers. Running it multiple times is safe.

`packages/domain/src/customers.ts`

Contains `listCustomersForUser(userId)`. The query scopes through an active employee instead of accepting a business ID from the browser.

`apps/web/app/customers/`

File	Purpose
<code>page.tsx</code>	Server-rendered customer list
<code>loading.tsx</code>	Skeleton shown while the route loads
<code>error.tsx</code>	Client-side retry card if loading fails

`package.json`

Adds `pnpm db:seed`, forwarding to the migration app's seed script.

Current result: visiting `http://localhost:3000/customers` shows Aisha Brown, Maria Lopez, and James Wilson from PostgreSQL.

How to start the current project demo

Use the focused two-terminal startup for a presentation. It avoids extra worker/Azurite noise and prevents accidental duplicate database servers.

Preparation — repository terminal

```
cd "C:\Users\Juanc\OneDrive\Desktop\Webdevelopment Fall\Fall Project\Clientflow_CRM-"  
git switch learning/company-customers  
pnpm install  
pnpm prisma:generate
```

Normally `pnpm install` is only needed after a fresh clone or dependency change.

Terminal 1 — database

```
pnpm db:dev
```

Wait for:

```
postgres ready – this process owns the data
```

Terminal 2 — seed

```
pnpm db:seed
```

Expected:

```
seeded demo business, locations, employee, and customers
```

Terminal 2 — web app

```
pnpm dev:web
```

Open these URLs

URL	What to show
<code>http://localhost:3000/api/health</code>	<code>{"status":"ok","db":"ok"}</code> proves the web process can reach PostgreSQL
<code>http://localhost:3000/customers</code>	The company-scoped customer list from the seed data

Stop cleanly

1. Press `Ctrl+C` in the web terminal.
2. Press `Ctrl+C` in the database terminal.

3. Wait for PostgreSQL's shutdown message before closing the window.

Avoid duplicate servers: do not run `pnpm dev`, `pnpm db:dev`, or `pnpm dev:web` twice. A second database process will fail on port 5433, and a second web process may move to port 3001.

Common local problems

Symptom	Likely cause	Action
P1001 or cannot reach 127.0.0.1:5433	PostgreSQL is not running	Start one <code>pnpm db:dev</code> and wait for “postgres ready”
Health returns 503 with <code>db:error</code>	Web is running but database is unavailable	Check port 5433 and the database terminal
“Port in use”	A duplicate dev/database process is already running	Stop old terminals; run only one stack
<code>fatal: not a git repository</code>	Terminal is one folder above the repository	<code>cd .\Clientflow_CRM-</code>
Customers page is empty	Seed was not run or user mapping differs	Run <code>pnpm db:seed</code> ; confirm current user is <code>demo-user</code>
Prisma types do not include new models	Generated client is stale	Run <code>pnpm prisma:generate</code>
Package-manager version/signature failure	Corepack/pnpm mismatch or registry unavailable	Use the repository-pinned pnpm 11.17.0 and confirm registry/network access

Focused commands

```
# Database only
pnpm db:dev

# Web only
pnpm dev:web

# Repeatable demo data
pnpm db:seed

# Prisma client after schema edits
pnpm prisma:generate

# Required checks before declaring merge-ready
pnpm test
pnpm typecheck
pnpm build
```

The full `pnpm dev` command starts web, worker, PostgreSQL, and Azurite together. Use it when demonstrating the whole platform, but only after all focused processes are stopped.

How to decide what to modify

Goal	Start here	Usually also update
Change page layout or text	<code>apps/web/app/...</code>	Feature spec if behavior changes
Add a form/API input	<code>packages/domain</code> Zod schema	Web action/route, tests, spec
Read database records	<code>packages/domain</code> query	User/business scope tests
Add a database field/table	<code>schema.prisma</code>	New migration, generated client, seed, tests, spec
Change current-user behavior	<code>packages/auth</code>	Auth spec and security tests
Add background processing	<code>apps/worker</code>	Queue adapter, message contract, tests
Add file/image storage	<code>packages/services</code>	Web endpoint, worker if processing, storage spec

Feature implementation ritual

- 1 Issue:** describe why, behavior sketch, out-of-scope items, and verification sketch.
- 2 Spec:** write observable behavior, examples, failures, constraints, and verification.
- 3 Implementation:** derive identity, validate, scope the query, act, map errors, and revalidate.
- 4 Tests:** include happy path plus a failure that would catch removal of the feature.
- 5 Review:** read the diff, run the complete checks, and verify the documented drill.

Database safety rules

- ✓ Every query is scoped from the current user.
- ✓ Foreign or unknown resources return 404, not 403.
- ✓ Never edit an applied migration; add the next numbered migration.
- ✓ Writes that imply history must save the record and history event transactionally.
- ✓ Do not edit or commit `.pgdata`, `.azurite`, or `.env`.
- ✓ Review generated SQL before applying it.

Recommended next work

Turn the read-only prototype into a complete customer creation slice only after the team reviews the tenant design.

- 1. Create the official issue and customer feature specification.
- 2. Add a Zod schema for customer input in `packages/domain`.
- 3. Add a scoped create function that derives `business_id` from the current user.
- 4. Build an accessible "Add customer" form.
- 5. Map validation errors to 400 and duplicate email/phone conflicts to 409.
- 6. Add tests with two businesses proving cross-tenant reads and writes are impossible.
- 7. Decide whether services, employee roles, and employees' location assignments must also be business-scoped now.

Glossary

Term	Meaning in this project
Tenant	An independent subscribing Business whose data must be isolated
Scope	The ownership condition added to a query, derived from the current user
Migration	An ordered SQL change that moves an existing database forward
Seed	Repeatable development/demo data
Domain layer	Shared web validation and database-query logic
Seam	A configuration boundary between a local stand-in and production service
ADR	A durable record of an architectural decision and its tradeoffs

You can safely explore: keep work on `learning/company-customers`, avoid pushing until ready, make small changes, run focused typechecks, inspect the diff, and reset only the local database when you intentionally want fresh data.

This guide describes the local prototype as it exists on the learning branch. Before merging, the repository requires an approved issue/spec workflow plus passing test, typecheck, build, and manual verification checks.